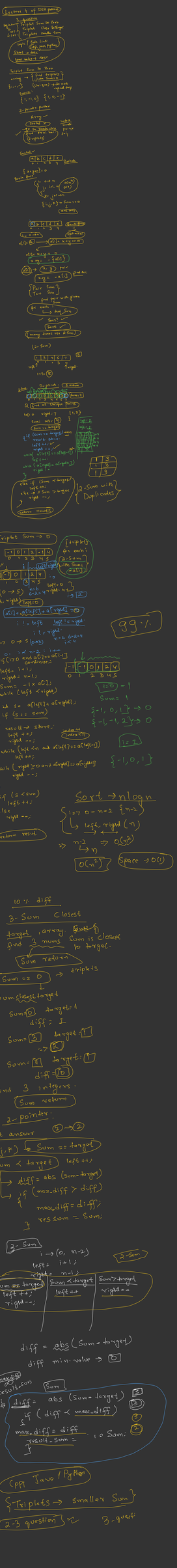

Lecture 4 of DSA patterns

3 question

Logic → 10% / 20%

- Triplet Sum to Zero.
- Triplet Close to Target
- Triplets Smaller Sum

Logic / Code Link. (Cpp, java, python)
Sheet → desc
Ipad notes → desc

Triplet Sum to Zero

array → { find triplets jinka Sum = 0 }

{-, -, -} (Unique) → do not repeat trip

Sum=0:
{1, -1, 0} {1, 0, -1}

2-pointer patter

Array ✓
Sorted ✗
✓ Ek se zaada chiz find krni hai. (triplets)

Index
number
pair ✗
trip

Sorted ✓

a	b	c	d	e
0	1	2	3	4

Duplicates

{x+y+z} = 0

Brute force

```
{ i = 0 → n
  { j = i+1, n
    { k = j+1 → n
```

O(n^3)
O(1)

{i, j, k} → Sum == 0 ↓ ans wey

i

a	b	c	d	e
0	1	2	3	4

Brute force → Optimised

i = 0 → n

a[i] = a → a[i] + x + y == 0

a[i] + x + y = 0

x + y = - {a[i]}

a[i] → x, y pair

x + y = - a[i] find this

{ Pair Sum / Two Sum }

find pair with given Sum

for each i → two Sum

✓ Sort? ✓

Sort ✓

(many times use 2-Sum)

(2-Sum)

1	3	4	5	7
0	1	2	3	4

left ↑ ↑ right. 9

1+7 = 8

2-Sum Duplicate : 1 answer

left left left left

1	1	1	2	2	3	3	3
0	1	2	3	4	5	6	7

Sum = 4

Q find all Unique Pairs

lef = 0 right = 7

Sum = 1+3 = 4

sum == target

{1, 3}
{

left = 1
left = 2

1	3	→ 4
1	3	→ 4
1	3	→ 4
1	3	→ 4
2	2	

1	3
1	3
1	3

```
{ if (sum == target) → ans mi
    result. store.
    left ++; ✓
    right --; ✓
    while( a[left] == a[left-1])
        left ++;
    while ( a[right] == a[right+1])
        right --
}
else if (Sum < target)
    left ++;
else → // Sum > target
    right --;
}
return result
```

{ 2-Sum with Duplicates }

Triplet Sum → 0

-1	0	1	2	-1	4
0	1	2	3	4	5

{ triplet } for each i { 2-Sum with Sum = -a[i] }

Sort

i ↓ | left/right

-1	-1	0	1	2	4
0	1	2	3	4	5

i →

n=6
6-2=4

left = 0
right = n-1 }

i → 2

i = (0 → 5)
(left, right) left = 0

a[i] + a[left] + a[right] = 0

i != left   left != right
i != right.

n=6 6-2=4
i < 4

i=> 0 → 5 (0→3)

```
{ i = 0 ; i < n-2 ; i++
    if (i>0 and a[i] == a[i-1]]
        continue;
    left = i+1;
    right = n-1;
    Sum = -1 x a[i];
    while (left < right)
    {
        int s = a[left] + a[right];
        if (s == sum)
        {
            result → store;
            left ++;
            right --;
            while (left < n and a[left] == a[left-1])
                left ++;
            while ( right ≥ 0 and a[right] == a[right+1])
                right --;
        }
        else if (s < sum)
            left ++;
        else
            right --;
    }
    return result
```

index ++
index < n

99%

↓

-1	-1	0	1	2	4
0	1	2	3	4	5

i = 0 = 1
Sum = 1
{-1, 0, 1} → 0
{-1, -1, 2} → 0

i = 1

{-1, 0, 1}

Sort → n log n

{ i => 0 - n-2 {n-2}
  → left, right (n)

=> n-2 → n => O(n^2)

O(n^2)   Space → O(1)

10% diff

3-Sum Closest

target, array. ~~find~~ {} find 3 nums Sum is closest to target.

Sum return

Sum == 0 → triplets
↓
Sum closest target

Sum = 0 target = 1
diff = 1

Sum = 3 target = 1
=> 2

Sum = 1 target = 1
diff = 0

find 3 integers.
Sum return

2-pointer.

2 → 2

Best answer

(i, j, k) → Sum == target

i
left
right
maxdiff
∅

Sum < target   left ++,

```
→ diff = abs (sum - target)
→ if (max_diff > diff)
  {
      max_diff = diff;
      res sum = Sum;
  }
```

2-Sum

i → (0, n-2)
left = i+1;
right = n-1;

2-Sum

Sum ~~==~~ target	Sum < target	Sum > target
left ++; right--;	left ++	right --

diff = abs (Sum - target)

diff min. value → 0

max diff ∅
result_sum   Sum

i left r

```
diff = abs (Sum - target)     5
if (diff < max_diff)          3
{                             3
max_diff = diff               2
  result_sum = ,6 Sum;
}
```

(Cpp / Java / Python)

{ Triplets → smaller Sum }

2-3 question } 3-questi